Conferencing on the Internet

P Cordell, M Courtenay, S Rudkin

This paper discusses three key requirements for conferencing applications — user or conference location, latency, and scalability. It then looks at the IETF and ITU approaches to meeting these requirements. Finally a protocol called SUCCESS, which attempts to combine the benefits of both approaches, is outlined.

1. Introduction

Any distributed real-time application operating over an Internet protocol (IP) network has to address the problems of quality of service, packet loss, jitter, and synchronisation of different media [1] . However, there are a number of key problems which are of particular significance to IP conferencing applications [1]:

- user or conference location — the discovery of the address of someone who is to be invited into the conference, or the discovery of the location of a conference that can be joined without invitation,
- latency — the end-to-end delay of control messages and conference media,
- scalability and robustness — the ability to robustly support widely varying requirements in terms of number of users, number of simultaneous conferences, geographical distribution of participants, etc.

This paper discusses these key requirements and approaches to their solution. It then looks at two different technologies which have been developed to support conferencing over IP. The first is the work of the Internet Engineering Task Force (IETF) whose main goal is scalable conferences. The second is the International Telecommunications Union (ITU) developed H.323 standard [2] which is aimed at supporting small, tightly controlled conferences. H.323 and IETF conferencing share a common approach to the delivery of media, but are significantly different in terms of how they control conferences. This paper illustrates these differences with particular reference to the above requirements.

[1] The term conferencing application is used in this paper to mean any multi-party interactive application (such as audio/video/data conferencing, networked games, shared virtual worlds, and electronic auctions.

2. Requirements and approaches

2.1 User and conference location

The first step in introducing a new member to a conference is the location of one party by another. This takes one of two forms depending on whether a user is invited into the conference (user location) or whether the user sees a conference announcement and then chooses to join the conference (conference location).

User location involves identifying the current IP address and port at which a given user can be contacted. If users did not move from terminal to terminal, did not have dynamically allocated addresses and were never unavailable this could be a straightforward domain name service (DNS) look-up. Unfortunately all these complications arise.

Dynamically allocated addresses and the fact that users change location can be handled by requiring users to register with a location server each time their circumstances change (this may be a user-driven or computer-driven action).

When a user is unavailable (intentionally or not), they are likely to want to use some kind of divert facility (e.g. divert all calls, divert on busy or divert on no-reply). Divert all calls can easily be handled at the location server. Divert on busy or divert on no reply are a little more difficult. Some possible solutions include the following.

- A server intervenes in the call set-up path to mimic the Private Branch Exchange (PBX) function — this allows the sequence of search steps to be specified local to the terminal being called. Unfortunately it introduces a possible single point of failure.

- The destination system implements a search of alternative systems in a similar way to the above. This removes the single, system-wide point of failure, but it does assume that the system is always switched on.
- The location service returns a divert on busy address and a divert on no-reply address at the same time that it provided the initial address. However, this requires an arbitrarily complex search algorithm (such as ring address 1 three times, then ring addresses 2 and 3 four times and so on) to be encapsulated in some form of protocol. It also means that the calling party must fully implement the intelligence needed to interpret the coded search algorithm.
- After failing to connect with the destination system, the calling system takes the initiative to contact the location service to explicitly ask for a divert on busy or divert on no-reply address.
- Multicasting a request for a terminal (perhaps via a location server) is particularly appropriate when it is a workgroup which is to be contacted rather than an individual. For the individual case, it works well for small numbers of users, but does not scale well to many terminals, as each terminal will be interrupted each time an invitation is made to any terminal. However, it may form part of an overall scheme in which a highly mobile set of people or a workgroup are contacted using multicast, and the rest are contacted using one of the above schemes. Alternatively, a user location server could try a multicast location after the user location server's search algorithm has failed to find the right person.

Conference location is about determining the IP address and port of a conference typically through a conference announcement. The announcement may be multicast, e-mailed or published (e.g. on a Web page). The conference itself may be multicast or unicast.

2.2 Latency

Latency can be a problem both for conference control and media delivery. This section looks at latency of media delivery and considers delays associated with one aspect of conference control, namely, set-up. We have chosen to look at set-up because this is an essential aspect of every conference.

Media delivery delay

A common scenario for current interactive services (e.g. Internet telephony) is where both users are connected to their Internet service providers (ISPs) by 28.8 kbit/s modems (see Fig 1). Often these modems are connected to the local computer via a serial line operating at 38.4 kbit/s. The user datagram protocol (UDP) [3], either unicast or multicast, is typically used for the delivery of media.

The consequences for the end-to-end delay are quite horrendous as can be seen in the third column of Table 1. Delays which may be pipelined are denoted in Table 1 by a bold left-hand boundary line. However, the situation can be improved significantly (as shown in the fourth column of Table 1) by adopting the following techniques:

- use low delay codec, e.g. G.729A [4],
- use pre-emptible priority scheduling and use a protocol processing architecture that allows the timing of the protocol processing to be under the control of the application (see, for example, Lee et al [5]),
- use header compression [6] to reduce the transmission delays,
- use an internal modem (to avoid communicating with the modem via a serial link),
- turn off modem's error checking, compression, and block transmission,
- use priority scheduling in the network (this minimises queuing delay and the need to buffer to account for jitter arising from any variation in network queuing delay).

Together these measures can reduce the delay substantially. The resulting end-to-end delay (calculated over 5000 km) would be just about satisfactory for telephony. Further improvements in the end-to-end delay could be achieved by using more powerful machines to speed up the encode/decode process by using a lower latency transmission technology such as ISDN or ADSL, or an even lower latency codec such as G.728. While G.728 is both more complex and uses a higher bandwidth (16 kbit/s) than G.723.1, processing power and (arguably) bandwidth will become less of an issue with time, whereas latency will continue to be a problem.

The biggest surprise in Table 1 is that for the chosen number of bytes of data, header compression has minimal effect on delay. This is because transmission/serialisation delays are masked by the modem processing delays. In the case of an internal modem, the size of the packet makes no difference until the transmission delay for a packet is at least 32 ms, i.e. the packet is at least 115 bytes long. Then each additional byte adds 2* 0.28 = 0.56 ms delay one way (@28.8 kbit/s).

Fig 1 A common scenario for today's Internet telephony users.

Table 1 Sources of delay in Internet telephony.

Source of delay	Calculation	Typical current delay	Achievable delay
Frame size (sampling time)	Determined by codec	30 ms (G.723.1 [7][1]	10 ms (G.729A)
Additional sound card/ API/os latency (over and above sampling time)		~30 ms (WAVE API, FIFO based protocol processing)	~5 ms (pre-emptible priority scheduling and real-time protocol processing)
Encode/decode	Look ahead + 2* 'frame-size'* required_MIPs/ available_MIPs	<67.5 ms (G.723.1)	< 25 ms (G.729A)
Packet header	Add 42 bytes	24 bytes becomes 66 bytes	10 bytes becomes 14 bytes (use header compression)
Serial link at transmitter	10/8* packet size/ line_speed	20 ms	0 (use internal modem)
Modem tx processing	53 symbols @ 3200 symbols/s	17 ms	17 ms
Modem waiting time	50 ms	50 ms	0 (turn off compression, error checking, transmission of blocks)
Modem transmission	packet/ modem_speed	18 ms	4 ms (fewer bytes)
Modem rx processing	103 symbols @ 3200 symbols/s	32 ms	32 ms
Propagation delay	5 ms per 1000 km	25 ms	25 ms
Router queuing delay	~ 10 ms per router	~ 100 ms	~ 10 ms (priority scheduling)
Modem tx processing	53 symbols @ 3200 symbols/s	17 ms	17 ms
Modem waiting time	50 ms	50 ms	0 (turn off compression, error checking, transmission of blocks)
Modem transmission	packet/ modem_speed	18 ms	4 ms (fewer bytes)
Modem rx processing	103 symbols @ 3200 symbols/s	32 ms	32 ms
Serial link at receiver	10/8 * packet size/ 38.4 ms	20 ms	0 (internal modem)
Buffer for network jitter, etc	2*mean variance in queuing delay	~ 200 ms	~ 20 ms (priority scheduling)
Buffer for os latency		30 ms	~ 5 ms pre-emptible priority scheduling
Total		~ 700 ms one way ~ 1.4 s both ways	~200 ms one way ~ 400 ms both ways

[1] The speech coding schemes G.723.1 and G.729(A) have the following delay characteristics:

	Look-ahead	Frame
G.729	5 ms	10 ms
G.723.1	7.5 ms	30 ms

In the absence of an internal modem, the size of the packet starts making an impact for packets with a transmission delay of at least 17 ms, i.e. when the packet is at least 61 bytes long. Then every additional byte adds 0.28 ms delay one way. Above 115 bytes every additional byte adds a delay of 3*0.28 ms = 0.86 ms one way. So, when used with 28.8 kbit/s modems as in the chosen scenario, header compression is only useful for any reduction in packet size it has above 61 bytes and is especially useful for the reduction it has above 115 bytes. This is in absolute terms. In relative terms, the longer the packet, the less significant is the saving of ~ 38 bytes. Also, above a certain packet size, the audio sampling time will no longer be hidden by the operating system latency in the API calls to the sound system.

For data interactions, the sound card and encode/decode delays are not relevant. However, play-out is dependent on the display frame rate, so the play-out delay would have to be increased to at least 15 ms. This would give a saving of about 20 ms in each direction resulting in a round-trip delay of around 360 ms.

Unlike audio frames (which describe the audio signal over a period of around 20 ms), each video frame represents the video signal at a particular instant in time — consequently the coding schemes do not impose a minimum delay. However, in practice most software implementations have coding delays of around 100 ms. This is only significant for frame rates above 10 frames per second. Below this rate each frame can be processed before receiving the next frame. With improved PC performance the video coding delay can be expected to decline steadily.

Set-up delay

Besides the actual transport of media (audio and/or video), there are key problems that any conferencing system must solve. The first of these (referred to here as call control) is getting remote parties into a conference. The second (media establishment) is deciding which combination of audio and video to use, and telling all the parties involved. Both call control and media establishment contribute to the total set-up delay.

There are two main schemes for setting up conferences — announcement-based and invitation-based. These are shown in Figs 2 and 3.

Typically when using announcement-based conferencing, call control and media establishment are combined. A party receives an announcement describing the conference including details of the media to be used and the conference address. There is normally no opportunity to negotiate the conference media types. The party simply sends a join message which includes address and port number. Subsequently it receives media from the

conference. In multicast conferences, the multicast join latency will depend on the multicast routeing protocol and the tree structure at the time of joining.

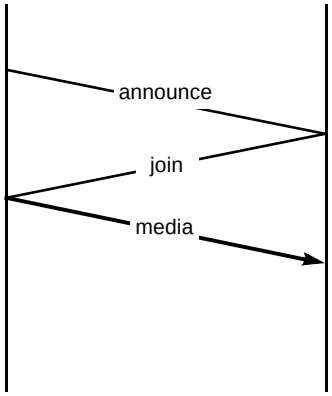


Fig 2 Announcement-based conferencing.

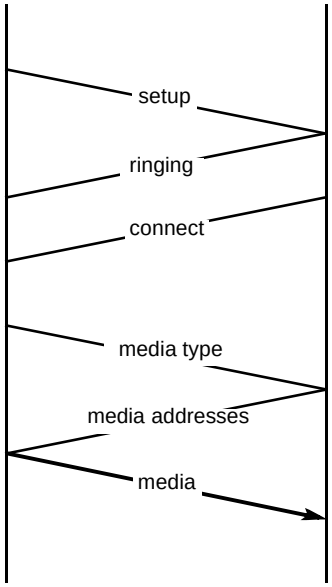


Fig 3 Invitation-based conferencing.

Invitation-based conferencing may or may not combine call control and media establishment into one phase. There are three main steps to call control. Firstly, an invitation is sent (this step involves locating the user to be invited). Secondly, the remote party sends an indication that it has received the invitation (this is important for the calling user so that they do not waste time waiting for a call that does not have sufficient resources at the far end for the call to take place). Thirdly, there is an indication that a user is on line[1]. In this paper, these three phases shall be referred to as *set-up*, *ringing* and *connected*.

[1] Connection can be reported by an explicit connection indication or by the arrival of media. An explicit indication has the advantage that it can be used to signal connection to other interested parties such as an intermediate server involved in call set-up (see section 2.1). The disadvantage is that it further complicates and slows down the call set-up process.

In general, the choice of media may be imposed by one of the parties or it may be negotiated. The former method is appropriate for announcement-based conferences or for invitations into large conferences. In these cases terminals containing many disparate capabilities may be present and so performing extensive capability negotiation can be prohibitively expensive.

The situation is different for small invitation-based conferences. The dynamic and unplanned nature of small conferences means it is important to be able to identify and use the best commonly supported codec.

Where media negotiation is required, it may be initiated within the call-control phase or it may follow in a separate media-establishment phase. Figure 3 shows the case where media negotiation follows call control and where the called party simply accepts the specified media type [2]. If the media and address messages were removed then it would represent the case where negotiation is initiated or the media is imposed in the initial set-up message. In this case the called party may then choose either to accept the call with the specified media type (as shown) or may offer alternative media types.

Clearly the set-up delay depends on the number of round trips involved, so in this respect, specification of the preferred media type in the set-up or announcement message is better than having a separate media establishment phase; it saves one round trip. Furthermore if implicit connection indication is used then negotiated media establishment introduces no extra delay — the invitee simply selects one of the specified media types and starts sending.

The other significant factor in determining delay is whether reliable connections must first be established for the signalling messages or whether soft-state is used. For example, when using the transport control protocol (TCP) [3], call set-up can be delayed by retransmissions, so these are only appropriate for very small conferences. Due to the TCP slow start, a packet lost during set-up can cost (in the case of the commonly referenced BSD implementation) 6 sec. Additionally, TCP incurs a one round trip time delay at start-up to synchronise the sequence numbers used for the acknowledgement process. This suggests UDP is preferable for conference control, although issues such as network friendliness, and accessibility to other protocols such as transport layer security (TLS) [8] also need to be considered, from a security point of view.

[2] The address message is only required for unicast media delivery. For multicast media delivery the address can be specified together with the media. However, a separate message is then required for the called party to join the conference.

2.3 Scalability and robustness

Scalability when applied to conferencing in Internet environments can mean a number of things.

- Can many small conferences be supported simultaneously as well as a few large ones?
- Can the required conference be located easily as the total number of choices increases?
- Can conferences with large resource and quality of service (QoS) requirements be supported as well as those with modest requirements?
- Can client, 'server' and network software run on equipment with modest resources as well as on equipment with plentiful resources?
- Can conferences allow participants to be widely spread geographically as well as based in the same locality?
- Can conferences support very large numbers of participants (e.g. public conferences) as well as small numbers?

Whether one situation can be supported 'as well as' another depends on a number of factors. Of particular significance are robustness, consistency, performance, reliability of delivery, and cost.

The Internet research and development community has two important mechanisms which can help with the need for scalability in conferencing — multicasting and soft-state control [9], the latter being specifically designed for robustness as well.

Multicasting is a mechanism provided by the network which assures only one copy of a particular packet is transferred across any link in the network, no matter how many receivers there may be. It relieves the sender of data from having to replicate that data to all receivers. As a result, data sources can send to 100s or 1000s of receivers without requiring a commensurate increase in resources (or decrease in the volume of data or its timeliness). Multicasting also provides a convenient abstraction of the set of all receivers of a particular data transmission, which can be shared by all senders. Further, multicast has the benefit that it allows the receiver to control what they receive without specific sender interaction.

Multicasting should not, however, be seen as the panacea for all scalability issues. For example, the efficient support of large numbers of small conferences would depend on the widespread deployment of multicast routeing protocols which do not place too great a load on routers in terms of supporting the state required. Multicast address allocation and multicast inter-service provider policy signalling — such as the border gateway protocol (BGP) — is also immature. There are also issues for dial-up customers. Currently, dial-up users are connected to boxes that contain multiple modems. Because of the way they currently operate, if one of the users connected to the box subscribes to a multicast address, then all users connected to the box will receive the stream. This could flood other users connections. It must be emphasised, however, that these are only short-term problems and are likely to be solved in the next year or two.

Soft-state control is exemplified in a number of Internet protocols (one of the most quoted examples is the resource reservation protocol — RSVP [10]). The basic principle is that control information is constantly refreshed; if a failure occurs which causes refreshes not to take place, the corresponding state will expire. Alternatively, if state is lost due to some failure, then it will be recovered the next time the control information is refreshed. Soft-state fits well with multicasting to maintain loose consistency of state in a scalable manner.

It is worth noting that stateless protocols have maximum robustness. The archetypal stateless protocol is the Internet protocol (IP) itself, which was designed specifically for robustness. The stateless hypertext transfer protocol (HTTP) v1.0 [11] has served the World Wide Web (WWW) well over a period of explosive growth, and the durability of IP is unquestionable. Stateless may, however, not be suitable for conferencing, which tends to exist over a defined period of time — hence soft-state is used. [1]

Multicasting can improve the efficiency and timeliness of both the delivery of the 'payload' of the conference (typically audio, video, collaborative data) and the control information associated with the conference. The former aspect has been well progressed by the Internet development community. The latter much less so, especially in the area of invitation-based conferencing. An issue is the lack of consensus about protocols to be used when multicast information must be reliably transferred from senders to receivers. However, in the absence of reliable multicast, soft state provides a very effective alternative.

Another reason for multicasting control information can be inferred from World Wide Web sites which have suddenly collapsed under a siege of overwhelming popularity. Sites supporting unicast control of live events could equally experience similar problems — particularly if multicast or any of the family of 'push' 'technologies are used to advertise the event. The result could be a 'control implosion' (Fig 4), resulting in severely degraded performance (customer dissatisfaction) or even system failure.

[1] Deficiencies in HTTP 1.0 leading to the definition of HTTP 1.1 with persistent connection are attributable to reliance on TCP (transmission control protocol).

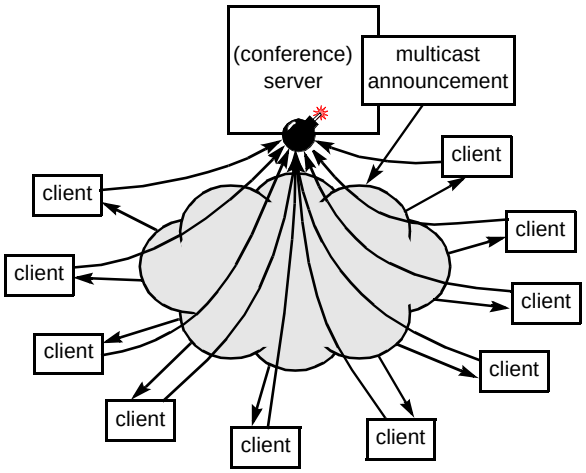


Fig 4 Conference control implosion.

A solution to the above dilemma is to design conference control protocols which are 'announcement friendly'. To date, such protocols have not been developed beyond very basic mechanisms, and this is potentially a significant area of future work. It should be stressed that this is still very much a research area, and, as described in section 4, an existing system — H.323 — has taken a more traditional and pragmatic approach to these issues. It remains to be seen whether the growth in multicast-based announcement and conferencing will justify confronting the technical challenges posed by distributed control.

3. Current work in the Internet Engineering Task Force

The IETF is an open organisation dedicated to developing engineering solutions, specifications and standards for the Internet and Internet-related technologies. An area which has been particularly successful is the development of a standard real-time transport protocol (RTP) [12] for use for real-time communication (including, of course conferencing) over packet networks. This specification has also been adopted by the ITU for the transport of audio and video in H.323 (see section 4). The IETF has also developed a number of 'payload formats' for RTP, which state how standard audio and video encodings developed within the ITU and ISO can be carried within the RTP framework.

The IETF has also pioneered the development of multicasting capability for the Internet and the use of soft-state protocols. RTP has been designed to be used in a multicast conference, though it also works well with unicast. A text-book example of a soft-state control protocol is the real-time control protocol (RTCP) [12] which is used as an adjunct to RTP. RTCP provides information about the senders and receivers of real-time streams in a conference, such as counts of packets received and lost. RTCP also provides basic conference membership information, which includes an indication of whether members of a conference have explicitly left the conference (the BYE message) or have stopped reporting due to some failure. RTCP has adopted a simple, but largely effective, approach to congestion control, which is to monitor the amount of bandwidth consumed within a conference and to adjust the retransmission interval accordingly to keep the overall bandwidth within a predetermined limit (5% of the RTP bandwidth). As a result, RTCP is highly scalable, though some concerns have been raised [13] about its scalability to huge conferences (number of receivers > 10 000). At this point, individual RTCP reports cease to be useful, and can place excessive demands on client resources to track them.

IETF activities on multicasting and conferencing are inextricably linked to the Mbone, an experimental test bed, established in the early 1990s, for multicast on a global scale achieved over the existing unicast-only Internet infrastructure by an overlay technique known as tunnelling. Key designers of RTP and associated control protocols have developed applications which are in wide use on the Mbone for conferencing and broadcasting of events and conferences such as the IETF's own meetings. More recently there has been activity within the IETF to deploy multicast within the wider Internet as an operational service, building on the success of the Mbone.

The IETF has also been addressing call and conference control in the form of the MMUSIC (multi-party multimedia session control) working group. A basic mechanism has been developed for publishing and announcing conferences, particularly for use with multicast. The session description protocol (SDP) [14] provides a general mechanism conveying the information required to participate in a conference, regardless of the transport by which that information is conveyed. The information conveyed specifies the types of media that shall be used, the person who originated the session, what the session is about, and so on. Such transports include e-mail, WWW, invitation or announcement (the latter mechanisms are described below). The session (conference) description includes information on each component media in the conference (see Fig 5).

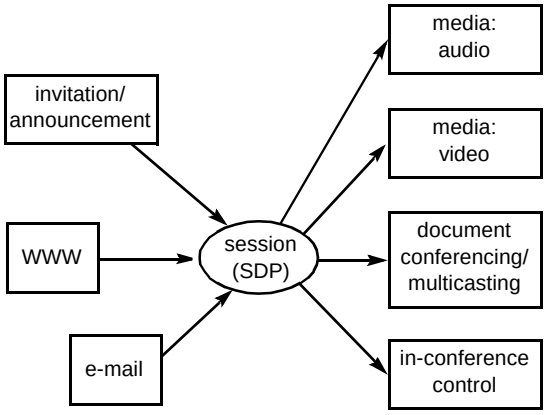


Fig 5 Scope of the session description protocol (SDP).

The MMUSIC real-time streaming protocol (RTSP) [15], a protocol to control on-demand real-time streams from multimedia servers, supports the invitation of a server to a conference by another mechanism (such as SIP or H.323), and hence allows the important scenario of introducing stored media to a conference.

Another important area for conferencing over the Internet is that of collaborative applications which do not necessarily have as great a requirement for low-delay delivery but may require reliable transfer of information. Such applications include point-to-multipoint file transfer or shared whiteboards. A number of reliable multicast protocols have been developed, notably SRM (scalable reliable multicast) [16] which is widely used on the Mbone in the form of the 'wb' shared whiteboard.

More recently, an IRTF (Internet Research Task Force) group has been investigating requirements and frameworks for reliable multicast protocols. The T.120 protocol suite [17] is being enhanced to exploit reliable multicast protocols for data delivery, though it currently does not specify the use of a particular protocol to achieve that reliability.

The following subsections describe in more detail the efforts of the IETF in the areas which are covered in this paper.

3.1 User location and invitation

The IETF has initially adopted an alternative mechanism to conventional user location for the establishment of conferences. The basis for this is a format for describing conferences (SDP). Session descriptions could potentially be distributed using e-mail (preferably well in advance), and, assuming that e-mail addresses of potential participants can be reliably obtained from directories and similar means, this could provide a satisfactory mechanism for setting up a conference. For the client, integration between e-mail, conferencing and calendaring/scheduling applications would be highly desirable to make this process more usable.

The IETF has also developed a session announcement protocol (SAP) [18] (see Fig 6), which involves transmitting session descriptions using multicast to a well-known address. This currently has experimental status within the IETF, though it is widely implemented and used on the Mbone. While the basic model for this is for use as a 'TV guide', it is also common for participants to use received SDP announcements to join a conference which has been notified by some other means, for example e-mail, chat, another conference in which the required participant is already taking part or even a conventional telephone call. In this case, it is not necessary to convey a complex session description, just knowing the subject matter or title of the conference is normally sufficient. Since the conference details should have already been 'pushed' to the required client using the announcement mechanism, there are often no delays in obtaining these.

These approaches are similar to the way traditional telephone audioconferences are typically set up. However, they could be further extended by embedding keywords in conference descriptions which will alert specific groups of people to participate in the conference, or the names of the required conference participants themselves. The latter is one of the functions proposed by the SUCCESS protocol [19].

The IETF has also developed the session initiation protocol (SIP) [20] for invitation. This has some overlap with the call control protocol defined by H.323 (see section 4). SIP packages an SDP announcement into a message which is sent using unicast to a remote party and includes acknowledgements so that the sender knows that the message has been received. The transport protocol used can be TCP (reliable) or UDP (datagram), the latter can use repetition of the request to overcome potential packet loss. Once a successful response has been received, it is then possible to proceed with other modes of communication (e.g. audio, video). Figure 6 illustrates the use of invitation and announcement to build a multicast conference.

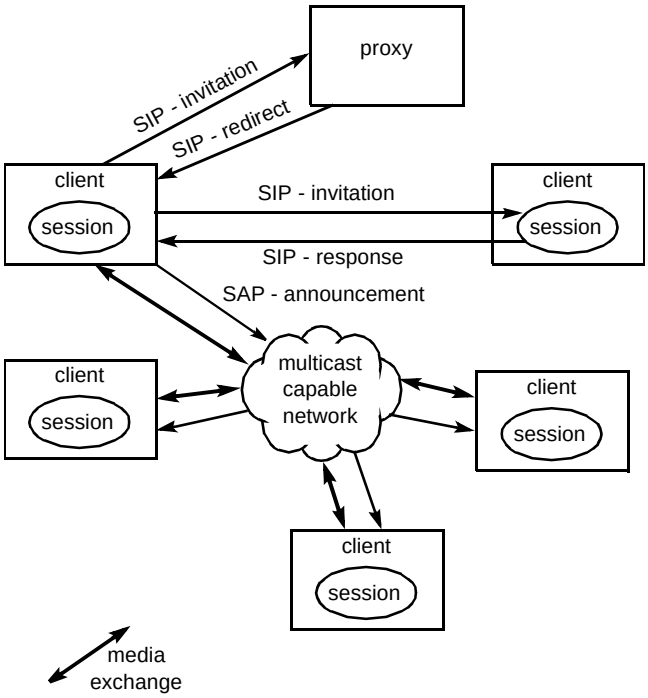


Fig 6 Building a conference with invitation and announcement

SIP takes a similar approach to user location to that used by the H.323 gatekeeper for user location. The basic assumption is that there may be a number of proxies which may know how to contact the invitee in question, in addition

to the invitee's client application itself. However, SIP allows for multiple simultaneous invitations to be sent and for multiple responses to be received by the initiating client itself, as shown in Fig 6. This potentially places a greater burden on the client, and may cause problems with adding value via intermediate servers.

3.2 Latency

'Connectionless' protocols may not necessarily provide any gains in terms of latency. For example, an IP packet traversing the network, when no previously cached routeing entries exist, will experience delays as potentially complex routeing decisions are made. If such a packet is multicast, the routeing is clearly more complex; initially, a multicast tree will have to be built the topology of which depends on a dynamic set of receivers.

Soft-state protocols often incur latency penalties. For example, the multicast membership protocol IGMP (Internet group management protocol) was initially defined to have no explicit leave mechanism, thus relying on time-outs to terminate streams from an unwanted conference. Membership information provided by RTCP will provide delayed indication of a particular participant leaving a conference in the event of the BYE message (the explicit signal of leaving) being lost. In conference control, a request which is multicast to a number of participants may have a delayed response as a result of random back-offs being applied to prevent implosion.

While the use of the datagram protocol UDP (especially in conjunction with multicast) provides for immediate delivery of control information for a conference, packet loss can cause delays corresponding to the retransmission interval used. Therefore there is a need to balance timeliness requirements with available bandwidth and interrupt burden when setting the retransmission frequency.

In summary, the design of protocols which exploit multicast and soft state can lead to many pitfalls in terms of latency. As a result, significant effort is required in the design of these protocols if it is important that latency is minimised.

3.3 Scalability

Scalability of conferencing to large numbers of participants has long been a focus in the development of IETF protocols, and indeed inspired the receiver-driven approach to conference membership that is provided by the IP multicast mechanism. The drivers for increased scalability in RTCP have come from the recognition that the technology being developed can potentially be deployed for the mass market. For conferencing, this could mean large scale debates, chat lines, etc, as well as networked games and virtual worlds. Anything but the most basic form of control for this sort of environment has, however, not been addressed in any detail in the IETF.

Another scalability issue relates to conference/session 'directories', which collect information about current conferences in progress. Clearly, if the number of conferences occurring simultaneously is large, then a single directory (which typically corresponds to a single multicast address and port) will not be suitable for carrying the associated announcements. There are two aspects to this:

- the congestion control back-off will cause an excessive retransmission interval, causing long delays in receiving session announcements,
- the user interface could become excessively cluttered.

The latter problem is less severe, as better user interfaces could be devised.

The issues of scalability of multicast and resource reservation referred to earlier (section 2) are actively being addressed in the IETF and have impact on applications other than conferencing.

4. H.323

H.323 [2] has been developed through the ITU. As with most Internet real-time multimedia applications, it uses the IETF's multicast-capable RTP for media transport. Where it differs from other protocols is in the connection set-up mechanisms, and how it decides which media to use.

H.323 version 1 was ratified in June 1996. Version 2 is currently being worked on and is due for ratification in January 1998. Much of the discussion below is applicable to both versions, although some of the features discussed will only be supported in version 2.

4.1 What does H.323 attempt to solve?

H.323 is aimed specifically at small conferences. An H.323 conference containing a few dozen people would be considered large. To support conferences larger than this, H.323 requires support from additional centralised conferencing equipment. As H.323 is intended to handle unplanned on-demand conferencing, this restriction is not unreasonable. It has to be admitted though that, for the largest of conferences, H.323 on its own is not the right solution. For this reason H.323 conferences can be combined with the IETF SDP protocol described above. This process is described in the ITU H.332 Recommendation [21].

Of the many requirements for H.323, one of the first is for interoperability with existing, non-Internet conferencing protocols, which includes H.320 [22] for use on the ISDN

and H.324 [23] for use on the PSTN. This is done by gateways. By virtue of interworking with H.320 and H.324, H.323 also interworks with basic voice services on the integrated services digital network (ISDN), and the public switched telephone network (PSTN).

Another requirement is that you should, as a minimum, be able to do with H.323 what you can do with a PBX. A PBX implicitly provides some support for user location. Sophisticated user search strategies, such as 'divert on no-reply', have an impact on the signalling needed, and H.323 has taken this into account.

4.2 The H.323 system

An H.323 system contains three different types of main entity. There are also some lesser components, and firewalls must be considered. Some of these entities are shown in Fig 7 which is shown from the perspective of a corporate intranet.

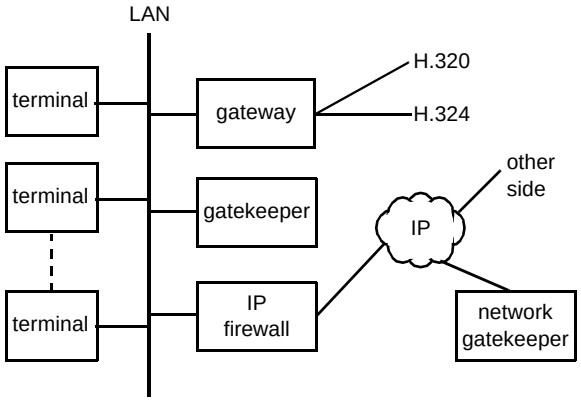


Fig 7 Main components of an H.323 system.

The first entity type is the terminal which users use to communicate with. Currently these are PC based, though there are indications that stand-alone terminals will appear in the future, but there are no known commercial offerings at the moment. These would replace the telephone on the corporate desk and connect directly to the local area network (LAN) via Ethernet.

The second entity type is gateways. These provide the bridging between the different networks such as H.323, H.324 and voice. Their use is optional in an H.323 system. Many vendors are in the final stages of developing H.323 gateways.

The third and final main entity is the gatekeeper. This is again optional in an H.323 system. However, it can be most useful as it collects all the features that cannot be easily distributed to the terminals or gateways, such as access control, telephone number to IP number translation, and user location.

Figure 7 actually shows two locations where gatekeepers can be used. The first of these is within the corporate environment and performs some or all of the functions listed above. An additional location for a gatekeeper is in a public network, such as the Internet or Concert Internet Plus. A gatekeeper in this location allows companies like BT to add value, possibly in the areas of directory services or 0800 number handling.

There are also some secondary entities which are not intended to be stand-alone, but which reside in the other main entities. One of these is the multipoint controller (MC) which will be discussed in section 4.5.

4.3 User location

H.323 uses the gatekeeper to act as the location server and to intervene in the call-set-up path in order to support 'divert on no-reply, and 'divert on busy' . Providing user location features in a local gatekeeper allows arbitrarily complex user search schemes to be implemented.

User location is based on terminals registering with the gatekeepers. Emulating PBX user location functions, such as 'divert on no-reply and busy', was a key requirement in the design of the H.323 call control. In this commonly used PBX service, the PBX attempts to contact a person at a primary number. If, after a suitable amount of ringing, nobody has answered, then an alternative number is tried. This sort of scheme requires signalling to go via the gatekeeper. Referring to the set-up, ringing, and connection indications mentioned in section 2, the gatekeeper needs to get a connected indication in addition to the calling terminal, so that it can stop the search algorithm. As the RTP media will be routed directly between terminals, and not via the gatekeeper, using the presence of RTP media to indicate that the parties are connected is not sufficient. Hence a connected message is required as part of the set of set-up messages.

When trying to emulate the divert on no-reply service, another indication also becomes apparent. This is an indication that the call has finished. This is used when the gatekeeper has been ringing the first terminal for a period and decides to try another terminal. This indication would tell the first terminal to stop ringing. Once again, this indication cannot rely on, say, the absence of RTP media as no RTP media has been sent at this time. Hence an additional message is required. Consequently the H.323 call control consists of messages associated with set-up, ringing, connected and closed, and does not rely on other events such as RTP to signal this.

4.4 Latency — H.323 call set-up times

H.323 supports invitation-based call-control and media negotiation. For reasons of interoperability with H.324,

H.320 and the PSTN, as well as the desire to reach the market quickly, H.323 uses separate protocols for the call control and media establishment phases. This removes the possibility of initiating media negotiation within the call control phase, resulting in an additional round-trip delay as shown in section 2. Furthermore H.245, which is used for media establishment, maintains a clear separation of capability transfer from the opening of logical channels. Therefore, even though the responder is in a position to select the media type once it has received the initiator's capabilities, it does not. The selection of media type, the allocation of resources and the transmission of media addresses are left for a separate message exchange. This results in a further half round-trip delay, Hence the total additional delay for media establishment is 1¾ round trips.

Another impact on set-up delays is that H.323 uses TCP to carry the call control and H.245 signalling channels (Fig 8). Consequently it suffers two additional round-trip delays (one for synchronising TCP sequence numbers for the call control channel and one for synchronising the H.245 channel). Also, both channels are subject to the TCP slow start and in particular the problems this presents if an initial packet is lost. If the initial packet is lost, a 6 second penalty (in the commonly referenced BSD implementation) is incurred. This figure only drops to about 3 seconds after half a dozen message exchanges. It must be emphasised that this delay is only incurred in the event of packet loss.

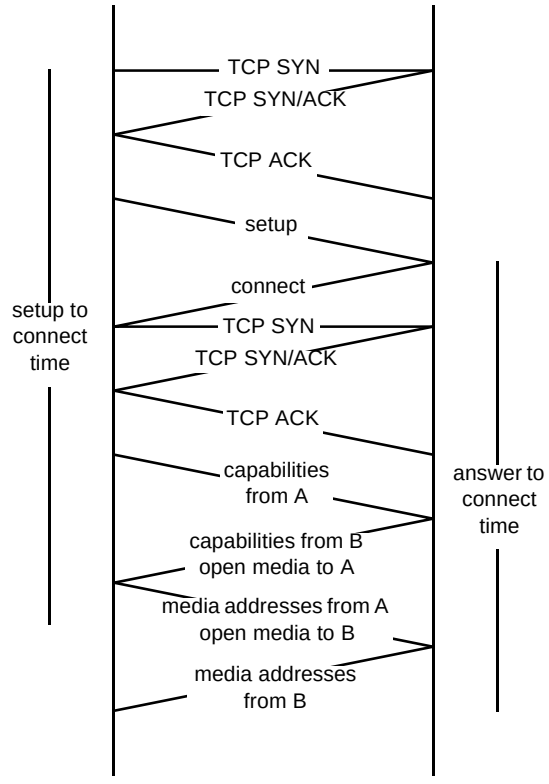


Fig 8 H.323 call set-up.

The resulting set-up times for H.323 calls are shown in Table 2. The round-trip times are calculated from the entries from Table 1 associated with signalling delay. The set-up to connected time consists of 4.5 round-trip times, which consists of 1 round trip for TCP SYNC for call control, 1 round trip for set-up and connected messages (assumes immediate answering), 1 round trip for H.245 TCP SYNC, and 1¾ round trips for media establishment. The 'answer to connected time' is similarly calculated, but starts from the point where the connect message is sent.

Table 2. Call set-up and answering times for various network delay times. (rtt = round-trip time. Figures for slow and fast dial-up based on the signalling delays from Table 1.)

	Slow dial up 1.2 s rtt	Fast dial up 360 ms rtt	Intranet 100 ms rtt
Set-up to connected with no ringing time	5.4 s	1.6 s	450 ms
Answering to connected with no ringing time	4.2 s	1.3 s	350 ms
Answering to connected with media establishment while ringing (see note 1)	600 ms	180 ms	50 ms

Note 1. Assumes that ringing time is longer than indicated set-up to connected time.

It is felt that users who can tolerate an audio round trip time of 1.4 s will have the patience to wait 5.4 s for call set-up. For shorter network delays, connection times are subjectively comparable with the existing switched network set-up times. (It should be noted that these figures are for a two-way call. An extra half round trip time will be incurred if an MC is involved as part of a multiparty call.)

4.5 Scalability and robustness

It has been mentioned that much of the signalling is point to point (media transport can make use of multicast, and hence is not an issue here). This presents a problem when communication between more than two parties is required. To cater for this, H.323 uses a multipoint controller, or MC. This acts as a signalling bridge for both the call control and H.245 signalling. The MC is responsible for terminating all of the point-to-point connections from the terminals involved. It receives all of the terminal's capabilities and decides which media should be used for this conference, and informs the terminals accordingly. This is a considerably simpler mechanism than relying on distributed decision making. Although this mechanism may not scale well (beyond, say, 20 nodes), this is consistent with H.323's design goals, and the mechanism is well understood as it is widely used in H.320 and H.324 conferencing. The functionality of the MC can reside in any of the three main entities (terminals, gateways and gatekeepers), and H.245 includes protocol for selecting the most appropriate MC when more than one is present in a call.

The down side of the MC approach is that it is functionality that should be present for the entire life of a call. This presents problems if the node containing the MC wishes to leave the conference, or the node containing the MC fails. The former situation can be eased by locating the MC in a centralised gatekeeper. This can remain active throughout a call, and handle many such calls simultaneously.

A greater problem is failure. There are two categories of failure mode from which centralised gatekeepers and MCs can suffer — hardware and software. In a client/server world, corporations are becoming accustomed to employing fault-tolerant hardware on servers, such as uninterruptible power supplies and redundant disk drives. Installing the gatekeeper (with associated MC) on such high MTBF systems would be an ideal choice.

The greater risk to reliability is often software failure. Software failure can be caused by bugs in the gatekeeper/MC code itself, the operating system that it runs on, or other, unrelated, processes causing the system to go unstable. To help with this, version 2 includes extensions that allow gatekeepers to be eliminated as a single point of failure. As part of the protocol, terminals and gateways are required to register with gatekeepers at regular intervals such as when they are switched on. As part of this process, the gatekeeper can provide backup network addresses in case a network card, or some other networking failure occurs. Indeed, the alternative network addresses given can be for different machines that are being operated in an unspecified redundant configuration that allows a hot-swap when a gatekeeper fails. The gatekeeper can also indicate alternative gatekeepers that are not being run in a redundant configuration, but that can provide sufficient support in the event that a terminal's primary gatekeeper fails.

MC failure should not be ignored as an issue either. However, a number of observations can be made. Firstly, the likelihood of failure is only weakly dependent on the number of terminals in the conference. Therefore, if 10 terminals are taking part in a conference the system is not 10 times more likely to fail. Although complicated, the MC's task is repetitive in nature. Hence freak operating conditions are unlikely, and the various paths through the code are likely to be well exercised. This means that the code is easier to test, and should mature faster in the field.

Also, if a remote terminal is using conformant messaging, only the MC needs to deal with it. The other terminals can be less tolerant to errant conditions and yet still function successfully.

Additionally, although the MC is present throughout the life of a conference, it only has a role to play when terminals enter or leave. If conference membership is stable, an MC failure should not affect the operation of the conference. Indeed, terminals may well remain oblivious to such an event.

5. SUCCESS

SUCCESS (simple universal call/conference establishment sequence) [19] is an attempt to combine the benefits of the IETF and ITU approaches. Thus it is designed to work well for announcement-based conferences and point-to-point calls that perhaps interwork with the ISDN/PSTN. It is fully distributed, and has the ability to use multicast for exchanging control information. The protocol has soft state features, but also has the ability to 'harden' the state, for example, when interworking with the ISDN/PSTN.

SUCCESS works by announcing that it is in a conference. This is done with a 'Hello' message, which is equivalent to the set-up indication mentioned in the overview section. The protocol also has a 'Progress' message which allows events like ringing to be conveyed. Where SUCCESS differs from other protocols is that the connected indication uses the same 'Hello' message used for the initial set-up indication. Thus, the calling terminal announces that it is in a conference to the called terminal, and the called terminal answers by announcing that it too is in the conference. So that control implosions do not occur when in a multicast environment, the 'Hello' message contains fields to indicate which terminal should respond to the message.

SUCCESS also has 'Bye' and 'ByeBye' messages to signal that an endpoint is leaving a conference. The 'Bye' message signals that a terminal is leaving the conference. The 'ByeBye' message is an optional acknowledgement of the 'Bye' message. Acknowledged 'Byes' are considered important when ISDN/PSTN interworking is in effect, and charges may be incurred.

Using these messages, SUCCESS can maintain elements of both tight and loose membership information for different terminals in the same conference.

At the present time SUCCESS is a paper design, but it is hoped to get some practical implementations running in the near future.

5.1 Scalability and robustness

SUCCESS is designed to be fully distributed. Part of the motivation for this is to achieve scalability and robustness. SUCCESS should easily cope with the largest of conferences as in this mode the protocol collapses back to being a simple announcement protocol like SDP. It has also been designed to work well in a point-to-point mode. These two special cases are fairly easy to model at the design stage. What will be interesting to see is how SUCCESS

handles evolving between these two extremes. Although this is part of the design goal, only implementation and exercising the protocol will truly see how well it adapts to conferences of different scale.

Another reason for making SUCCESS fully distributed is to remove dependence on any potential single point of failure. A consequence of being distributed is that SUCCESS uses soft state to periodically signal that it is still in the conference. Compared to always signalling to a central point, this periodic signalling imposes an extra burden on terminals. In a similar vein, it is also important that SUCCESS is network friendly. It is intended that the frequency with which SUCCESS refreshes itself should adapt to the prevailing conditions. This is complicated by the fact that some terminals may be on low-speed or highly asymmetric links. Achieving the desired characteristics while the conference is changing size represents an interesting challenge for the future.

5.2 Latency

Set-up latency was not a key design goal of SUCCESS, but low complexity was. Hence SUCCESS can be low latency. As part of this, it combines the call control and media establishment signalling within the same protocol. Being designed to exploit multicast, it uses UDP thus avoiding the signalling delays associated with TCP.

SUCCESS can also short cut set-up latency due to the way it supports media negotiation. The 'Hello' message contains two separate fields to indicate the capabilities that a terminal (or an entire conference) can receive and the set of media that the terminal is sending. A short cut can be achieved by including in the capabilities the addresses to which media should be sent if that capability is selected. Thus, for example, different audio codecs can be assigned to different ports, or RTP payload types, and the receiver can use this to select the appropriate decoder when the RTP packets arrive.

This allows the set-up latency to be effectively reduced to half a round-trip time. However, this approach does place additional burden on the receiver which not all platforms will be able to support. In this case the protocol falls back to the standard sequence of stating what can be received, what is to be sent, and what addresses are to be used.

6. Conclusions

Although H.323 does not represent the ultimate Internet real-time multimedia communications tool, it can be seen that it is most applicable for small conferences, especially those that take place over low-delay networks. Although it does suffer from an MC being a single point of failure, this is common to many other protocols, and there are well understood mechanisms for handling this.

Larger conferences are better serviced using the IETF's SDP protocol.

To provide some support for conferences that require a mixture of loose and tight coupling, the ITU has defined H.332 which describes how H.323 and SDP can be operated together in the same conference.

In the longer term , a single protocol such as SUCCESS [20] which is able to cope with all of the various conference models may be the way forward.

Acknowledgements

The authors would like to thank colleagues at BT Laboratories for their comments.

References

1 Rudkin S, Grace A and Whybray M: 'Real time applications on the internet', BTTJ 15, No 2 pp 209—225 (April 1997).

2 ITU-T Recommendation H.323: 'Visual telephone systems and equipment for local area networks which provide a non-guaranteed quality of service', (May 1996).

3 Braden R: 'Request for Comments: 1122, requirements for Internet hosts — communication layers', (October 1989).

4 ITU-T Recommendation G.729 — Annex A: 'Coding of speech at 8 kbit/s using conjugate structure algebraic-code-excited linear-prediction (CS-ACELP) Annex A: Reduced complexity 8 kbit/s CS-ACELP speech codec', (November 1996).

5 Lee C, Yoshida K, Mercer C and Rajkumar R: 'Predictable communication protocol processing in real-time mach', in: 'Proc of Real-time Technology and Applications Conference', IEEE Computer Society Press, pp 220—229 (June 1996).

6 Casner S and Jacobson V: 'Compressing IP/UDP/RTP headers for low-speed serial links', IETF Internet Draft draft-ietf-avt-crtp-01.txt (November 1996).

7 ITU-T Recommendation G.723: 'Dual rate speech coder for multimedia communications transmitting at 5.3 and 6.3 kbit/s', (March 1996).

8 Dierks T and Allen C: 'The TLS protocol transport layer', Security Working Group draft-ietf-tls-protocol-03.txt (May 1997).

9 O'Neill A W: 'Internetwork futures', BTTJ 15, No. 2, pp 226—240 (April 1997).

10 Braden R et al: 'Resource ReSerVation protocol (RSVP) —— Version 1 functional specification', IETF, RSVP working group work in progress (draft-ietf-rsvp-spec-14) (November 1996).

11 Berners-Lee T, Fielding R, Irvine U C, Frystyk H: 'Hypertext transfer protocol — HTTP/1.0, Request for Comments: 1945', MIT/LCS (May 1996).

12 Schulzrinne H, Casner S, Frederick R and Jacobson V: 'RTP: a transport protocol for real-time applications', RFC1889, IETF (January 1996).

13 Aboba B: 'Alternatives for enhancing RTCP scalability', Microsoft Corporation, draft-aboba-rtpscale-02.txt (January 1997).

14 Handley M, and Jacobson V: 'SDP: session description protocol', draft-ietf-mmusic-sdp-03, IETF (March 1997).

15 Schulzrinne H: 'A real time stream control protocol (RTSP')', IETF Internet Draft ietf-mmusic-stream-00.txt (November 1996).

16 Floyd S, Jacobsen V, McCanne S, Liu C G and Zhang L: 'A reliable multicast framework for light-weight sessions and application level framing', Proc ACM SIGCOMM, Cambridge, Mass (1995).

17 Russ M: 'Desktop conversations — the future of multimedia conferencing', BT Technol J, 15, No 4, pp 42—50 (October 1997).

18 'Announcement Protocol', IETF MMUSIC Working Group Internet Draft, draft-ietf-mmusic-sap-00.txt (November 1996).

19 Cordell P: 'SUCCESS — simple universal call/conference establishment sequence', BT internal document (Feb 1997).

20 Handley M, Schulzrinne H and Schooler E: 'SIP: session initiation protocol', IETF MMUSIC Working Group Internet Draft, ietf-mmusic-sip-02 (March 1997).

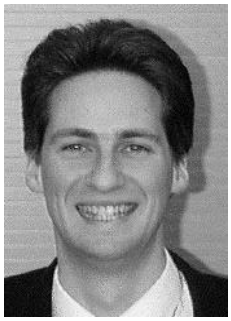
21 ITU-T Recommendation H.332: 'Visual Telephone Systems and Equipment for Local Area Networks', Work in progress.

22 ITU-T Recommendation H.320: 'Narrow-band visual telephone systems and terminal equipment', (March 1996).

23 ITU-T Recommendation H.324: 'Terminal for low bit rate multimedia communication', (March 1996).

Pete Cordell joined BT in 185 with a first in Electronics from the University of Sussex. He spent two years working on slow scan TV systems after which he was sponsored to do a PhD in Recursive Binary Nesting Moving Picture Coding at Heriot-Watt University, Edinburgh. He returned in 1990 to work on low-power ISDN telephones, and the VC8000 PC videotelephony system for which he had system level design and audio codec design responsibilities. Since then he has been a key contributor to the H.323 standard, and has been actively involved in a number of Internet-based conference protocols, including inventing the SUCCESS protocol.

Mark Courtenay graduated from the University of Surrey in 1982 with a first class honours degree in Electrical and Electronic Engineering. He worked on digital line systems for BT's trunk network in the City of London before completing an MSc in communications engineering at Imperial College and moving to BT Laboratories in 1985. Since then he has worked on software implementation and data networking protocol architectures, initially X.25 and, more recently, Internet protocols and multimedia communications systems, his current specialism being call control and multicast protocols for the Internet.



Steve Rudkin graduated from the University of Cambridge in 1985 with a BA in Electrical and Natural Sciences. After completing an MSc in Computation at the University of Oxford in 1986 he joined the SEMA group to work on real-time embedded software.

He moved to BT Laboratories in 1987 to research the use of formal and object-oriented methods in distributed systems. Since 1993 he has led the distributed systems group and its research into Web-based commerce, and multimedia on the Internet.